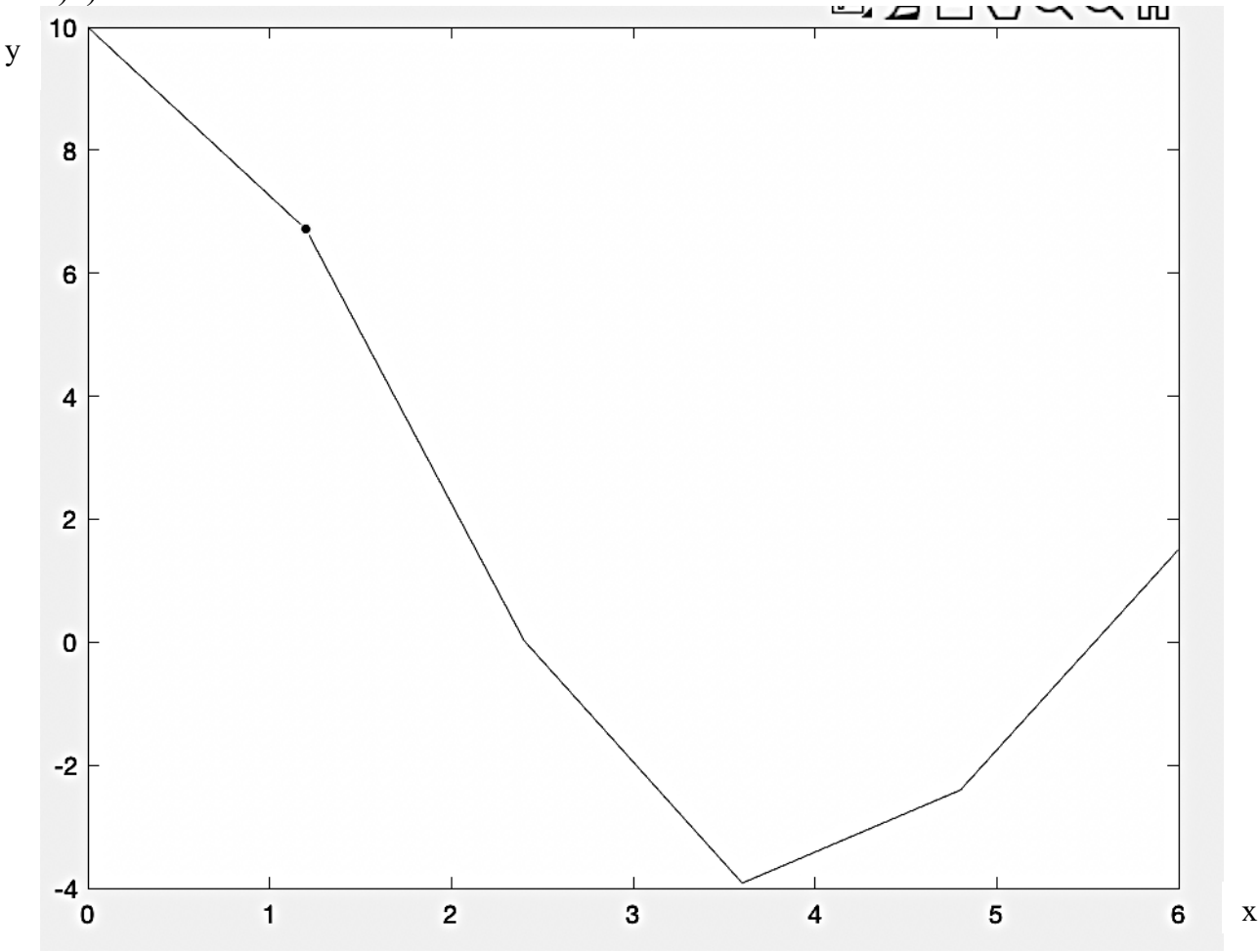
Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

MAT 214

NÜMERİK YÖNTEMLER

ÖDEV 1

1)a)



Kod:

```
Editor - /Users/fuad/Documents/MATLAB/proje1/a.m
+4 | secant_method.m | calculate_lagrange_error.m | calculate_newton_raphson_error.m | minus.m | a.m | +
1  xvalues = [0 1.2 2.4 3.6 4.8 6];
2  yvalues=[10.0000 6.7113 0.0251 -3.9177 -2.4043 1.5065];
3  plot(xvalues, yvalues );
```

b) Lagrange interpolasyon polinomu şu şekilde bulunur:

$L_{n,k} = \prod_{\substack{i=0 \\ i \neq k}}^{n} \frac{(x - x_i)}{(x_k - x_i)}$ olmak üzere

Lagrange interpolasyon polinomu = $P_n(x) = \sum_{k=0}^{n} L_{n,k}(x) * f(x_k)$

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

Soruda 5.dereceden Langrange interpolasyon polinomu istenilmektedir

O zaman Langrange İnterpolasyon polinomu aşağıdaki gibi olur:

$$P_5(x) = L_{5,0}(x) * f(x_0) + L_{5,1}(x) * f(x_1) + L_{5,2}(x) * f(x_2) + L_{5,3}(x) * f(x_3) + L_{5,4}(x) * f(x_4) + L_{5,5}(x) * f(x_5)$$

Matlaba implemente ederken $L_{n,k}$ fonksiyonunu x değerelerini alarak hesaplayan bir fonksiyon eklendi ve x 'i symbol olarak aldım böylece x değişken gibi gözükmeyecektir. Böylece P interpolasyon polinomu hesaplanırken her toplamda L fonksiyonu n ve k ya göre hesaplanıp kullanılmaktadır.
Matlab symbol değerlerini çarpıp toplayıp denkelemi sadeleştirmesi için expand() fonksiyonunu kullanıldı (langrangeip.m dosyasında)

Sonra main.m içinde x ve y vektörlerini tanımladım ve
```
lpoli = lagrangeip(xvalues, yvalues , 5);
```
çağırdım böylece fonksiyon hesaplanıp lpoli değişkenine atanmaktadır.
lpoli yaani lagrange interpolasyon polinomu sonucu bu şekilde bulundu:

P(x) = - (23435*x^5)/2985984 + (12589*x^4)/497664 + (95759*x^3)/138240 - (1287349*x^2)/345600 + (101441*x)/144000 + 10

Sayıların bölünmediğini farkettim ve bunun için ne yapabileceğimi araştırıldı Sayıların kesirli değil de bölünmesi için vpa() fonksiyonu kullanılabileceği bulundu. İlk parametre fonksiyon ve ikinci parametre virgülden sonra basamak sayısı olmak üzere bu fonksiyon kullanabilir ama çözünürlük düşüşü olmaması adına fonksiyon içinde kullanmamayı tercih edildi ve sadece main.m de denemeler yapmak amaçlı bırakıldı.

Virgülden sonra 10 basamak çözünürlükle bölmeler yapılırsa:
P(x) = - 0.007848334084*x^5 + 0.02529618377*x^4 + 0.6927010995*x^3 - 3.724968171*x^2 + 0.7044513889*x + 10.0
şeklinde bulunur.

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

Kodları :

Editor – lagrange.m

main.m | bisection.m | calculate_newton_raphson_error.m | lagrange.m

```
function [outputArg1] = lagrange(xvalues,n,k)
%LAGRANGE Summary of this function goes here
%   xvalues and yvalues are arrays
%   n is the degree of function
syms x real;

result = 1 ;
for i = 1:n+1 %I add 1 to k because in matlab when we parse arrays we start with 1 
    if k+1 ~= i
        result = result * ( (x-xvalues(i))/ (xvalues(k+1) - xvalues(i))  );
    end
end
outputArg1 = result;

end
```

Editor – lagrangeip.m *

main.m | bisection.m | calculate_newton_raphson_error.m | lagrange.m | lagrangeip.m *

```
function [outputArg1] = lagrangeip(xvalues, yvalues ,n)
%lagrangeip Summary of this function goes here
%   Detailed explanation goes here
syms x real;
interpolationp = 0;
for k= 0:n
    interpolationp = interpolationp + ...
        lagrange(xvalues , n , k) * yvalues(k+1);

outputArg1 = expand(interpolationp); % expand function able us to multiply
% all and summ all the similiar terms

end

```

Aşağıdaki kod main.m içindedir

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

main.m | bisection.m | minus.m | a.m | b.m | c.m | +

```
1   xvalues = [0 1.2 2.4 3.6 4.8 6];
2   yvalues=[10.0000 6.7113 0.0251 -3.9177 -2.4043 1.5065];
3   lpoli = lagrangeip(xvalues, yvalues , 5);
4   disp(lpoli);
5   lpoli = vpa(lpoli , 10);
6   disp(lpoli);
```

c)

$$n_k = \prod_{i=0}^{k-1}(x - x_i)$$

= newton çarpanı
Ve
$a_n = f[x_0, x_1, \dots, x_n]$
=bölünmüş farklar olmak üzere
(Köşeli parantezler bölünmüş farklara ait bir tanımdır)
Ve $f[x_i\ , x_{i+1}\ , \dots, x_{i+k}]$bu şekilde tanımlanırsa

$f[x_i] = f(x_i)\ \ ve$

$$f[x_i\ , x_{i+1}\ , \dots, x_{i+k}] = \frac{f[\ x_{i+1}\ , \dots, x_{i+k}] - f[\ x_{i+1}\ , \dots, x_{i+k-1}]}{x_{i+k} - x_i}$$

Newton interpolasyon polinomu şu şekilde bulunur:

$$N(x) = \sum_{i=0}^{k} a_k n_k(x)$$

a bölünmüş farklar katsayısını bulmak için `dividedDiferences()`
fonksiyonunu yazıldı bu fonksiyon recursive bir şekilde a sayısını hesaplamaktadır.
Ve n çarpanını bulmak için newton adlı fonksiyonu yukardaki tanıma uyacak şekilde yazdım ve x'i symbol olarak tanımlaandı .
Sonra interpolasyon polinomu N(x) i bulmak için newtonip fonksiyonu yazıldı
Bu fonksiyonun içinde Newton ve dirivedDiferences fonksiyonlarına çağırarak genel fonksiyon hesaplandı ve exapand() fonksiyonuyla denklem sadeleştirildi.
main.m de Newton interplasyon polinomu bu şekilde hesaplatırıldı

```
npoli =newtonip(xvalues,yvalues,5);
```

ve sorunun cevabı ise şu şekilde bulundu:
N(x) = - (1131064142526445*x^5)/144115188075855872 +
(24607558901461501*x^4)/972777519512027136 +

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

(374357809650495791*x^3)/540431955284459520 -
(2516364790232543299*x^2)/675539944105574400 +
(297428157421933753*x)/422212465065984000 + 10

Virgülden sonra 10 basamak çözünürlüğüyle bölmeler yapılırsa
N(x) = - 0.007848334084*x^5 + 0.02529618377*x^4 + 0.6927010995*x^3 -
3.724968171*x^2 + 0.7044513889*x + 10.0
sonucu elde edilir.

Kodları :

Editor – dividedDiferences.m *

Variables – fxpoli_f_secant_t

main.m | bisection.m | calculate_newton_raphson_error.m | lagrange.m | lagrangeip.m | dividedDiferences.m *

```
function [outputArg1] = dividedDiferences(xvalues,yvalues)
%DİVİDEDDİFERENCES Summary of this function goes here
%   Detailed explanation goes here
if length(xvalues) ==1
    result = yvalues(1);
else
    result = (dividedDiferences(xvalues(2:end),yvalues(2:end)) ...
        - dividedDiferences( xvalues(1:end - 1),yvalues(1:end - 1) ) )/...
        (xvalues(end) - xvalues(1) );
end
outputArg1 = result;
end
```

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

Editor – newton.m

main.m | dividedDiferences.m | newton.m

```
function [outputArg1] = newton(xvalues,k)
%newton Summary of this function goes here
%   Detailed explanation goes here
syms x real;
result = 1;

for i=1:k-1
    result = result * (x - xvalues(i));
end
outputArg1 = result;
end
```

Editor – newtonip.m

main.m | dividedDiferences.m | newton.m | newtonip.m

```
function [outputArg1] = newtonip(xvalues,yvalues, n)
%NEWTONİP Summary of this function goes here
%   Detailed explanation goes here
func = 0;
for i = 1:n+1
    func = func + (dividedDiferences(xvalues(1:i) ,yvalues(1:i)) ...
        * newton(xvalues,i));
end

outputArg1 = expand(func);

end
```

main.m | bisection.m | minus.m | a.m | b.m | c.m | +

```
1   xvalues = [0 1.2 2.4 3.6 4.8 6];
2   yvalues=[10.0000 6.7113 0.0251 -3.9177 -2.4043 1.5065];
3   npoli =newtonip(xvalues,yvalues,5);
4   disp(npoli);
5   npoli = vpa(npoli,10);
6   disp(npoli);
```

d)
kodu :

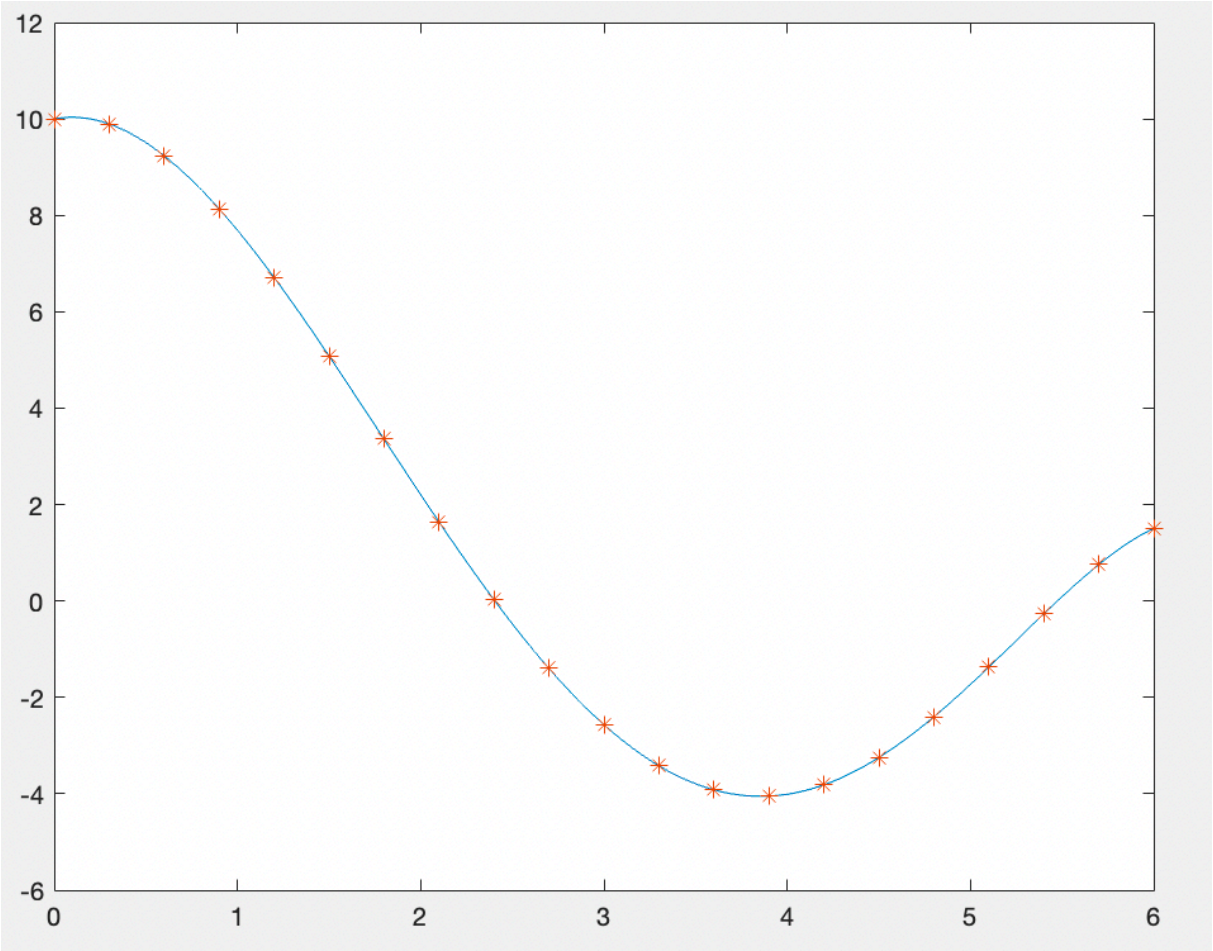
Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

main.m | bisection.m | minus.m | a.m | b.m | c.m | d.m | +

```
1   xvalues = [0 1.2 2.4 3.6 4.8 6];
2   yvalues=[10.0000 6.7113 0.0251 -3.9177 -2.4043 1.5065];
3   lpoli = lagrangeip(xvalues, yvalues , 5);
4   npoli =newtonip(xvalues,yvalues,5);
5   lpoli_f = matlabFunction(lpoli);
6   npoli_f = matlabFunction(npoli);
7   ax = axes;
8   plot(xvalues,yvalues);
9   x_cnt = 0:0.0001:6;
10  y_cntl = lpoli_f(x_cnt);
11  plot(x_cnt,y_cntl);
12  hold on
13  y_cntn = npoli_f(x_cnt);
14  plot(x_cnt,y_cntn,'*','MarkerIndices',1:3000:length(y_cntn))
15  hold off
```

Not : çizgi olan fonksiyon langrange interpolasyon fonksiyonu ve yıldızlı olan ise Newton interpolasyon fonksiyonu , ayırt edilebilmesi için bu şekilde gösterilmiştir.

y Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808



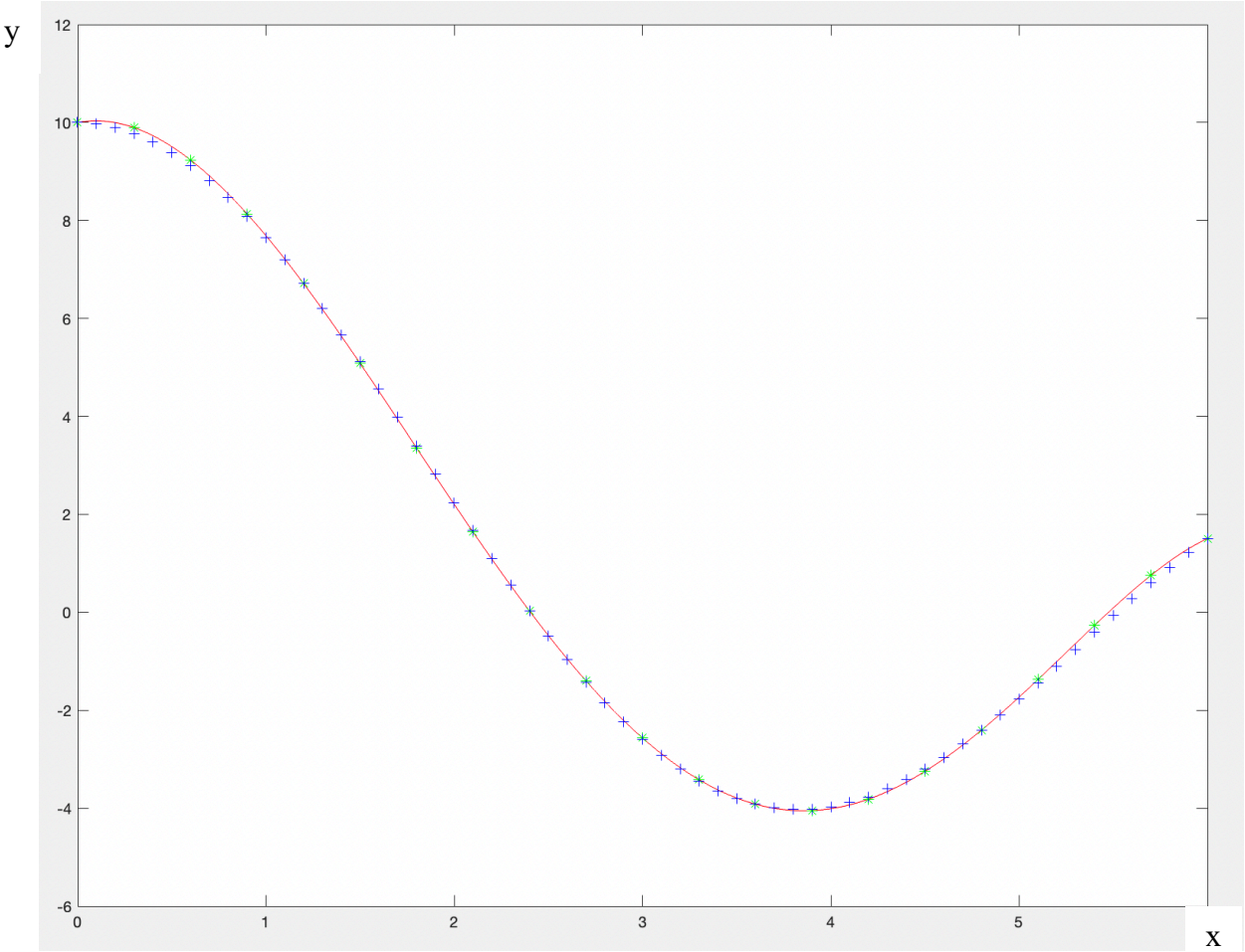
e) Kod:

Editor - /Users/fuad/Documents/MATLAB/proje1/e.m

main.m | bisection.m | minus.m | a.m | b.m | c.m | d.m | e.m

```
xvalues = [0 1.2 2.4 3.6 4.8 6];
yvalues=[10.0000 6.7113 0.0251 -3.9177 -2.4043 1.5065];
lpoli = lagrangeip(xvalues, yvalues , 5);
npoli =newtonip(xvalues,yvalues,5);
lpoli_f = matlabFunction(lpoli);
npoli_f = matlabFunction(npoli);
ax = axes;
plot(xvalues,yvalues);
x_cnt = 0:0.0001:6;
y_cntl = lpoli_f(x_cnt);
plot(x_cnt,y_cntl);
hold on
y_cntn = npoli_f(x_cnt);
plot(x_cnt,y_cntn,'*','MarkerIndices',1:3000:length(y_cntn));
y_exact = 10 * besselj(0, x_cnt);
plot(x_cnt,y_exact,'+','MarkerIndices',1:1000:length(y_cntn));
ax.ColorOrder = [1 0 0; 0 1 0; 0 0 1 ]; %the first plot will be red , the
hold off
```

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808



Not : burda çizgi olan fonksiyon langrange interpolasyon fonksiyonu, yıldızlı olan ise Newton interpolasyon fonksiyonu ve artı ile çizilen fonksiyon 10*besselj fonksiyonudur, ayırt edilebilmesi için bu şekilde gösterilmiştir.

f) Eğer değerler $f(x) = 10 * J_0(x)$ uyması gerekiyorsa ve interpolasyon polinomumuza P(x) denilirse o zaman mutlak hata

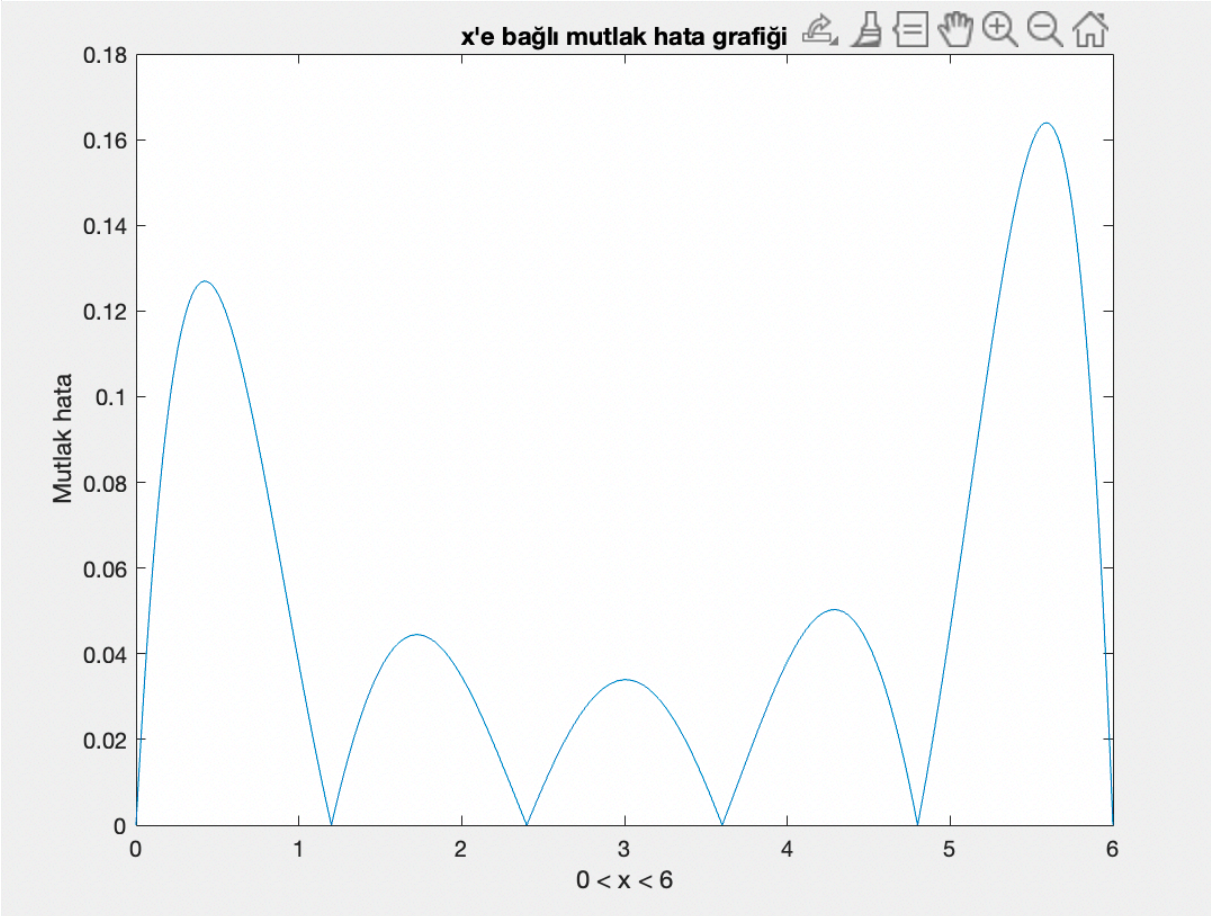
$$R(x) = |\big(P(x) - f(x)\big)|$$

Kod:

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

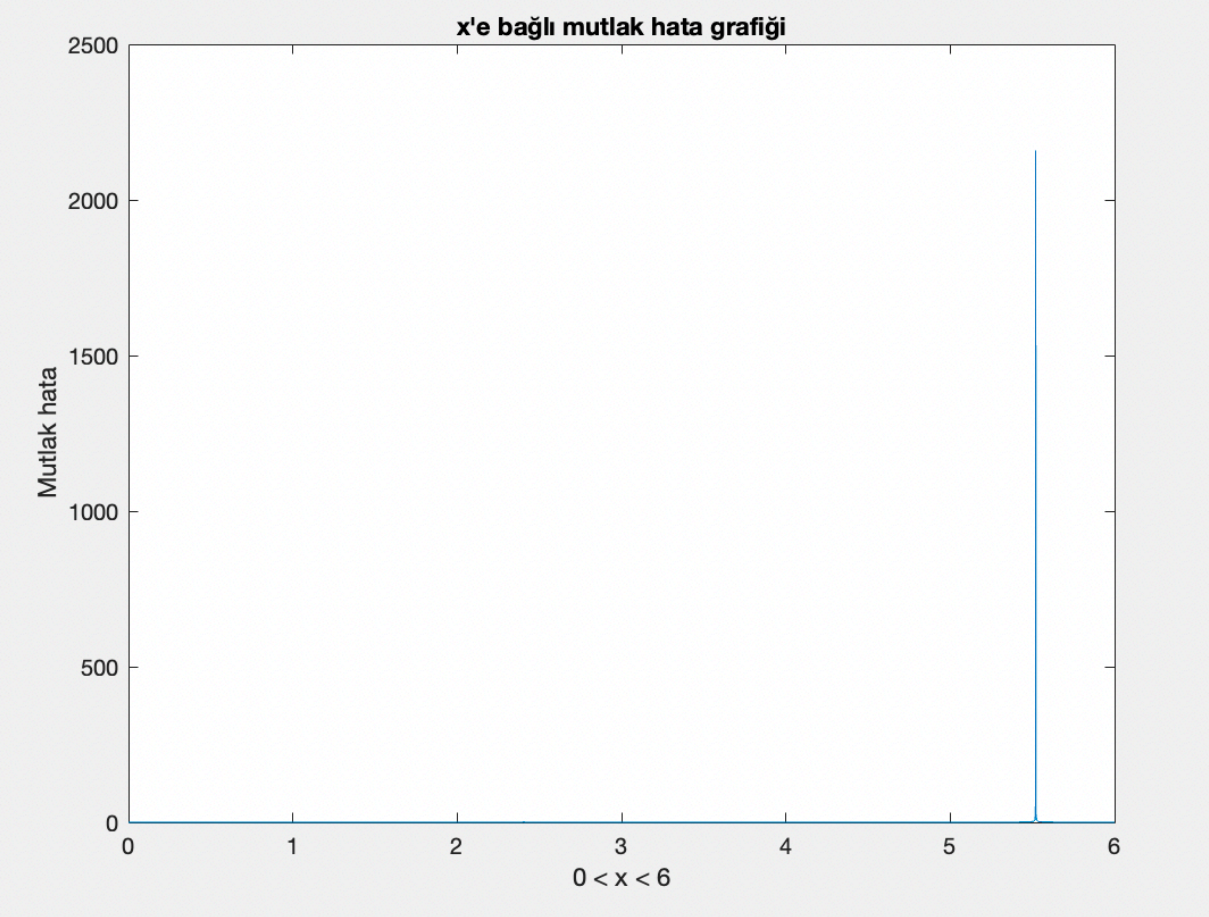
```
xvalues = [0 1.2 2.4 3.6 4.8 6];
yvalues=[10.0000 6.7113 0.0251 -3.9177 -2.4043 1.5065];
lpoli = lagrangeip(xvalues, yvalues , 5);
lpoli_f = matlabFunction(lpoli);
x_cnt = 0:0.0001:6;
y_cntl = lpoli_f(x_cnt);
y_exact = 10 * besselj(0, x_cnt);
%hold on
%plot(x_cnt ,y_cntl );
%plot(x_cnt,y_exact);
error = abs((y_cntl - y_exact) )  ;
abs_error = [];
for i=1:1:length(error)
    abs_error(end+1) = abs(error(i)/y_exact(i));
end

plot(x_cnt , error )
title("x'e bağlı mutlak hata grafiği")
xlabel('0 < x < 6')
ylabel('Mutlak hata')
```



Veya

$R(x) = |\frac{(P(x)-f(x))}{f(x)}|$ şeklinde tanımlarsak



Ve kodu şu şekildedir:

main.m | bisection.m | minus.m | a.m | b.m | c.m | d.m | e.m | f.m | +

```
xvalues = [0 1.2 2.4 3.6 4.8 6];
yvalues=[10.0000 6.7113 0.0251 -3.9177 -2.4043 1.5065];
lpoli = lagrangeip(xvalues, yvalues , 5);
lpoli_f = matlabFunction(lpoli);
x_cnt = 0:0.0001:6;
y_cntl = lpoli_f(x_cnt);
y_exact = 10 * besselj(0, x_cnt);
%hold on
%plot(x_cnt ,y_cntl );
%plot(x_cnt,y_exact);
error = abs((y_cntl - y_exact) )  ;
abs_error = [];
for i=1:1:length(error)
    abs_error(end+1) = abs(error(i)/y_exact(i));
end
plot(x_cnt,abs_error);
title("x'e bağlı mutlak hata grafiği")
xlabel('0 < x < 6')
ylabel('Mutlak hata')
%plot(x_cnt , error )

%hold off
```

g) d şıkkında intepolasyon polinomunda ve gerçek fonksiyona bakılırsa
x = [2 , 3] aralığında ve x = [5,6] aralığında bir kök bulunduğu gözüküyor .

h)Genel kök bulma kodları:

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

```
function [outputArg1,outputArg2,outputArg3,outputArg4] = bisection(f,interval , iteration_number)
%UNTITLED Summary of this function goes here
%   Detailed explanation goes here
a_all = [];
b_all = [];
p_all = [];
fp_all = [];
a = interval(1);
%a_all(end + 1) = a;
b = interval(2);
%b_all(end + 1) = b;
for i = 1:iteration_number
    p = (a + b)/2;
    a_all(end + 1) = a;
    b_all(end + 1) = b;
    p_all(end + 1) = p;
    fp_all(end + 1) = f(p);
    %xline(p);

    if f(a) * f(p)  < 0
            b = p ;
    elseif f(b) * f(p) < 0
            a = p ;
    end

end
a_all(end + 1) = a;
b_all(end + 1) = b;
p_all(end + 1) = p;
fp_all(end + 1) = f(p);

outputArg1 = a_all;
outputArg2 = b_all;
outputArg3 = p_all ;
outputArg4 = fp_all;
end
```

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

main.m | bisection.m | newton_raphson.m

```m
function [outputArg1,outputArg2] = newton_raphson(f,p0,iteration_num)
%NEWTON_RAPHSON Summary of this function goes here
%   f is symbolic function
f_n = matlabFunction(f);%fn is numeric function
df = diff(f);
df_n = matlabFunction(df);

p = p0;
p_all = [p0];
fp_all = [f_n(p0)];
for i= 1:iteration_num
    p = p - (f_n(p) / df_n(p));
    p_all(end + 1) = p;
    fp_all(end+1) = [f_n(p)];
end
outputArg1 = p_all;
outputArg2 = fp_all;
end

```

main.m | bisection.m | newton_raphson.m | fixed_point.m

```m
function [outputArg1,outputArg2] = fixed_point(fx,p0,iteration_number)
%FİXED_POİNT Summary of this function goes here
%   Detailed explanation goes here
p_all = [];
fp_all = [];
p = p0;
for i = 1:iteration_number
    p_all(end+1) = p;
    fp_all(end + 1) = fx(p);
    p = fx(p);
    %xline(p);
end
outputArg1 = p_all;
outputArg2 = fp_all;
end


```

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

Editor - secant_method.m

main.m | bisection.m | newton_raphson.m | fixed_point.m | secant_method.m

```
function [outputArg1,outputArg2] = secant_method(f, p0 ,p1 ,iteration_num)
%SECANT_METHOD Summary of this function goes here
%   f is matlab function (numeric)

p_all =[p0 p1];
fp_all = [f(p0) f(p1)];
for i=3:iteration_num
    p = p_all(i-1) - ( ( f(p_all(i-1)) * ( p_all(i-1) - p_all(i-2) ) )/...
        (f(p_all(i-1)) - f(p_all(i-2)) ) ) ;
    p_all(end + 1) = p;
    fp_all(end + 1) = f(p);
end
outputArg1 = p_all;
outputArg2 = fp_all;
end
```

+2 | newton_raphson.m | fixed_point.m | secant_method.m | calculate_lagrange_error.m | calculate_newton_raphson_error.m

```
function [outputArg1,outputArg2] = calculate_newton_raphson_error(f,p0,p,inteval)
%CALCULATE_NEWTON_RAPHSON_ERROR Summary of this function goes here
%   f is symbolic function
result = (diff(diff(f))/2) * ((p - p0)^2);
delta = 0.0001;
func_result = matlabFunction(result);
value_of_result = [];
for i=inteval(1):delta:inteval(2)
    value_of_result(end + 1 ) = func_result(i);
end
numeric_result = max(value_of_result);

outputArg1 =numeric_result ;
outputArg2 = result;

end

```

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

1.kök için kodlar:

```
xvalues = [0 1.2 2.4 3.6 4.8 6];
yvalues=[10.0000 6.7113 0.0251 -3.9177 -2.4043 1.5065];
lpoli = lagrangeip(xvalues, yvalues , 5);
npoli =newtonip(xvalues,yvalues,5);
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
lpoli_f = matlabFunction(lpoli);
[l_a_it_b,l_b_it_b,l_p_it_b,l_fp_it_b] = bisection(lpoli_f,[2,3],20);
lpoli_f_bis_t = table(l_a_it_b.',l_b_it_b.',l_p_it_b.',l_fp_it_b.');
l_poli_bis_err  =( l_b_it_b(end) - l_a_it_b(end)) / 2;

[l_p_it_f , l_fp_it_f] = fixed_point(lpoli_f,2.5,20);
lpoli_f_fixed_t = table(l_p_it_f.',l_fp_it_f.');

[l_p_it_n , l_fp_it_n] = newton_raphson(lpoli,2.5,6);
lpoli_f_newtonr_t = table(l_p_it_n.',l_fp_it_n.');
[l_raph_error ,l_raph_error_func] = calculate_newton_raphson_error...
(lpoli , l_p_it_n(end-1) , l_p_it_n(end), [2.4 , 2.5]);

[l_p_it_s , l_fp_it_s] = secant_method(lpoli_f,2,3,20);
lpoli_f_secant_t = table(l_p_it_s.',l_fp_it_s.');
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
npoli_f = matlabFunction(npoli);
[n_a_it_b,n_b_it_b,n_p_it_b,n_fp_it_b] = bisection(npoli_f,[2,3],20);
npoli_f_bis_t = table(n_a_it_b.',n_b_it_b.',n_p_it_b.',n_fp_it_b.');
npoli_bis_err  =( n_b_it_b(end) - n_a_it_b(end)) / 2;

[n_p_it_b , n_fp_it_b] = fixed_point(npoli_f,2.5,20);
npoli_f_fixed_t = table(n_p_it_b.',n_fp_it_b.');

[n_p_it_n , n_fp_it_n] = newton_raphson(npoli,2.5,4);
npoli_f_newtonr_t = table(n_p_it_n.',n_fp_it_n.');
[n_raph_error ,n_raph_error_func] = calculate_newton_raphson_error(lpoli ,
n_p_it_n(end-1) , n_p_it_n(end), [2.4 , 2.5]);

[n_p_it_s , n_fp_it_s] = secant_method(npoli_f,2,3,20);
npoli_f_secant_t = table(n_p_it_s.',n_fp_it_s.');
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
syms x real;
fx_poli = 10 * besselj(0,x);
fx_poli_f = matlabFunction(fx_poli);

[f_a_it_b,f_b_it_b,f_p_it_b,f_fp_it_b] = bisection(fx_poli_f,[2,3],20);
fxpoli_f_bis_t = table(f_a_it_b.',f_b_it_b.',f_p_it_b.',f_fp_it_b.');
fx_poli_bis_err  =( f_b_it_b(end) - f_a_it_b(end)) / 2;

[p_it , fp_it] = fixed_point(fx_poli_f,2.5,20);
fxoli_f_fixed_t = table(p_it.',fp_it.');

[p_it , fp_it] = newton_raphson(fx_poli,2.5,4);
fxpoli_f_newtonr_t = table(p_it.',fp_it.');
[fx_raph_error ,fx_raph_error_func] = calculate_newton_raphson_error(...
   lpoli , p_it(end-1) , p_it(end), [2.4 , 2.5]);

[p_it , fp_it] = secant_method(fx_poli_f,2,3,9);
fxpoli_f_secant_t = table(p_it.',fp_it.');
```

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

1.kök için

1)Lagrange interpolasyon polinomu için

1.1)Bisection yöntemi ile kök bulma :

n	a_n	b_n	p_n	$f(p)_n$
0	2	3	2.50000000000000	-0.474774614917559
1	2	2.50000000000000	2.25000000000000	0.813401750564577
2	2.25000000000000	2.50000000000000	2.37500000000000	0.153466957632814
3	2.37500000000000	2.50000000000000	2.43750000000000	-0.164971381487323
4	2.37500000000000	2.43750000000000	2.40625000000000	-0.00678809448387696
5	2.37500000000000	2.40625000000000	2.39062500000000	0.0730859864383273
6	2.39062500000000	2.40625000000000	2.39843750000000	0.0330848901527041
7	2.39843750000000	2.40625000000000	2.40234375000000	0.0131322973964743
8	2.40234375000000	2.40625000000000	2.40429687500000	0.00316806555764337
9	2.40429687500000	2.40625000000000	2.40527343750000	-0.00181102478510908
10	2.40429687500000	2.40527343750000	2.40478515625000	0.000678267974128843
11	2.40478515625000	2.40527343750000	2.40502929687500	-0.000566441529572970
12	2.40478515625000	2.40502929687500	2.40490722656250	5.58974438877868e-05
13	2.40490722656250	2.40502929687500	2.40496826171875	-0.000255275987768755
14	2.40490722656250	2.40496826171875	2.40493774414063	-9.96902581302805e-05
15	2.40490722656250	2.40493774414063	2.40492248535156	-2.18966536653653e-05
16	2.40490722656250	2.40492248535156	2.40491485595703	1.70003334751812e-05
17	2.40491485595703	2.40492248535156	2.40491867065430	-2.44817550409948e-06
18	2.40491485595703	2.40491867065430	2.40491676330566	7.27607513439921e-06
19	2.40491676330566	2.40491867065430	2.40491771697998	2.41394885058810e-06
20	2.40491771697998	2.40491867065430	2.40491771697998	2.41394885058810e-06

Bisection yönteminin mutlak hatası bu şekilde hesaplanır

$$|p_n - p| \leq \frac{b_n - a_n}{2}$$

O zaman

$$|p_n - p| \leq 4.768371582031250e-07$$

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

1.2)Fixed point yöntemi ile (p0 = 2.5 alındı tahmini aralığın yarısı(g şıkkına bakınız))

n	p_n	$f(p)_n$
0	2.50000000000000	-0.474774614917559
1	-0.474774614917559	8.75323798526763
2	8.75323798526763	-59.4590178638101
3	-59.4590178638101	5990013.13280748
4	5990013.13280748	-6.05223970948188e+31
5	-6.05223970948188e+31	6.37320788780027e+156
6	6.37320788780027e+156	NaN

Bir noktaya yaklaşmak söz konusu olmadığından kök bulunamadı.

1.3)Newton raphson metodu ile (p0 = 2.5 alındı tahmini aralığın yarısı)

n	p_n	$f(p)_n$
0	2.50000000000000	-0.474774614917559
1	2.40279199206967	0.0108447942490102
2	2.40491725418038	4.77344394766988e-06
3	2.40491819046024	9.30810983845731e-13
4	2.404918190460423	-3.55271367880050e-15
5	2.404918190460422	1.77635683940025e-15
6	2.404918190460423	0

Newton metodunun hata payı ,
epsilon tahmini kök aralığında([2.4,2.5])(fonksiyon aralığı almamıza gerek yoktur çünkü p0 = 2.5 verdik ve p_n 2.4 ve 2.5 aralığında olduğunu görüyoruz) bir sayı olmak üzere şu şekilde bulunur

$$R(p_n) = \frac{f''(\varepsilon)}{2} * (p_n - p_{n-1})^2$$

Bu hatayı bulmak için calculate_newton_raphson_error adında bir fonksiyon yazıldı içinde R(p) hata polinomunu bulundu sonra aralık sınırları içinde delta = 0.0001 olmak üzere epsilon örneklenip R(x)'in maximum olduğu nokta bulundu.

$$R(p) = -\frac{117175 * x^3}{75703688197437786823974636480028999 68} + \frac{12589 * x^2}{42057604554132103791097020266682777 6} + \frac{95759 * x}{23365335863406724328387233481490432 0} - \frac{1287349}{17524001897555043246290425111117824 00}$$

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

$$\max\big(R(p)\big) = \quad 2.3520e-31$$

1.4)Sekant metodu ile (p0 =2 , p1 = 3 alındı, tahmini aralığın sınırları)

n	p_n	$f(p)_n$
0	2	2.20423113854595
1	3	-2.56658398437500
2	2.46202401094017	-0.287600849383132
3	2.39413305157952	0.0551085111331737
4	2.40505008543040	-0.000672422615746626
5	2.40491848382453	-1.49566014684410e-06
6	2.40491819045239	4.09716705007668e-11
7	2.404918190460422	8.88178419700125e-15
8	2.404918190460424	-7.10542735760100e-15
9	2.404918190460423	0

Secant yönteminin hatası ε hata olmak üzeri bu şekide hesaplanabilir

$$|r_{n+1} - r_n| \leq \varepsilon$$

O zaman $\varepsilon = \pm\ 7.1\text{e}-15$

Newton interpolasyon polinomu için kod :
bu kodları sırasıyla çalıştırarak Newton interpolasyon polinomunun değerlerini hesapladım ve tablolara aldım (`a_it,b_it,p_it,fp_it` aynı olduğundan kullanırken diğer satırları yoruma aldım)

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

2)Newton interpolasyon polinomu için

2.1) Bisection yöntemi ile kök bulma :

n	a_n	b_n	p_n	$f(p)_n$
0	2	3	2.50000000000000	-0.474774614917560
1	2	2.50000000000000	2.25000000000000	0.813401750564578
2	2.25000000000000	2.50000000000000	2.37500000000000	0.153466957632816
3	2.37500000000000	2.50000000000000	2.43750000000000	-0.164971381487327
4	2.37500000000000	2.43750000000000	2.40625000000000	-0.00678809448387696
5	2.37500000000000	2.40625000000000	2.39062500000000	0.0730859864383273
6	2.39062500000000	2.40625000000000	2.39843750000000	0.0330848901527023
7	2.39843750000000	2.40625000000000	2.40234375000000	0.0131322973964760
8	2.40234375000000	2.40625000000000	2.40429687500000	0.00316806555764337
9	2.40429687500000	2.40625000000000	2.40527343750000	-0.00181102478511264
10	2.40429687500000	2.40527343750000	2.40478515625000	0.000678267974128843
11	2.40478515625000	2.40527343750000	2.40502929687500	-0.000566441529572970
12	2.40478515625000	2.40502929687500	2.40490722656250	5.58974438877868e-05
13	2.40490722656250	2.40502929687500	2.40496826171875	-0.000255275987768755
14	2.40490722656250	2.40496826171875	2.40493774414063	-9.96902581320569e-05
15	2.40490722656250	2.40493774414063	2.40492248535156	-2.18966536653653e-05
16	2.40490722656250	2.40492248535156	2.40491485595703	1.70003334716284e-05
17	2.40491485595703	2.40492248535156	2.40491867065430	-2.44817550232312e-06
18	2.40491485595703	2.40491867065430	2.40491676330566	7.27607513439921e-06
19	2.40491676330566	2.40491867065430	2.40491771697998	2.41394885058810e-06
20	2.40491771697998	2.40491867065430	2.40491771697998	2.41394885058810e-06

Bisection yönteminin mutlak hatası bu şekilde hesaplanır

$$|p_n - p| \leq \frac{b_n - a_n}{2}$$

O zaman

$$|p_n - p| \leq 4.768371582031250e-07$$

2.2)Fixed point yöntemi ile (p0 = 2.5 alındı tahmini aralığın yarısı(g şıkkına bakınız))

n	p_n	$f(p)_n$
0	2.50000000000000	-0.474774614917560
1	-0.474774614917560	8.75323798526762
2	8.75323798526762	-59.4590178638096
3	-59.4590178638096	5990013.13280723
4	5990013.13280723	-6.05223970948061e+31
5	-6.05223970948061e+31	6.37320788779356e+156
6	6.37320788779356e+156	NaN

Bir noktaya yaklaşmak söz konusu olmadığından kök bulunamadı.

2.3) Newton raphson metodu ile (p0 = 2.5 alındı tahmini aralığın yarısı)

n	p_n	$f(p)_n$
0	2.50000000000000	-0.474774614917560
1	2.40279199206967	0.0108447942490084
2	2.40491725418038	4.77344395122259e-06
3	2.40491819046024	9.25481913327531e-13
4	2.40491819046042	0

$$R(p_n) = \frac{f^{"}(\varepsilon)}{2} * (p_n - p_{n-1})^2$$

$$R(p) = -\frac{19601151175 * x^3}{75703688197437786823974636480028999 68} + \frac{2105900509 * x^2}{42057604554132103791097020266682777 6} + \frac{16018661279 * x}{23365335863406724328387233481490432 0} - \frac{215349028069}{17524001897555043246290425111117824 00}$$

Ve

$$\max\big(R(p)\big) = 3.934413099098205e - 26$$

2.4)Sekant metodu ile (p0 =2 , p1 = 3 alındı, tahmini aralığın sınırları)

n	p_n	$f(p)_n$
0	2	2.20423113854595
1	3	-2.56658398437500
2	2.46202401094017	-0.287600849383130
3	2.39413305157952	0.0551085111331684
4	2.40505008543039	-0.000672422615739521
5	2.40491848382453	-1.49566015217317e-06
6	2.40491819045239	4.09734468576062e-11
7	2.40491819046042243357	3.55271367880050e-15
8	2.40491819046042332175	-5.32907051820075e-15
9	2.40491819046042287766	0

Secant yönteminin hatası ε hata olmak üzeri bu şekide hesaplanabilir

$$|r_{n+1} - r_n| \leq \varepsilon$$

O zaman $\varepsilon = \pm 5.32907051820075\text{e} - 15$

fx interpolasyon polinomu için kod :
bu kodları sırasıyla çalıştırarak fx interpolasyon polinomunun değerlerini hesapladım ve tablolara aldım (`a_it,b_it,p_it,fp_it` aynı olduğundan kullanırken diğer satırları yoruma aldım)

3) f(x) polinomu için
3.1) Bisection yöntemi ile kök bulma :

n	a_n	b_n	p_n	$f(p)_n$
0	2	3	2.50000000000000	-0.483837764681979
1	2	2.50000000000000	2.25000000000000	0.827498512887340
2	2.25000000000000	2.50000000000000	2.37500000000000	0.155783734620732
3	2.37500000000000	2.50000000000000	2.43750000000000	-0.168456529803729
4	2.37500000000000	2.43750000000000	2.40625000000000	-0.00739276482216946
5	2.37500000000000	2.40625000000000	2.39062500000000	0.0739378802025111
6	2.39062500000000	2.40625000000000	2.39843750000000	0.0332073408094519
7	2.39843750000000	2.40625000000000	2.40234375000000	0.0128908822279104
8	2.40234375000000	2.40625000000000	2.40429687500000	0.00274494459455180
9	2.40429687500000	2.40625000000000	2.40527343750000	-0.00232494022259895
10	2.40429687500000	2.40527343750000	2.40478515625000	0.000209744856419509
11	2.40478515625000	2.40527343750000	2.40502929687500	-0.00105766204018874
12	2.40478515625000	2.40502929687500	2.40490722656250	-0.000423974678071470
13	2.40478515625000	2.40490722656250	2.40484619140625	-0.000107118931985526
14	2.40478515625000	2.40484619140625	2.40481567382813	5.13119569749008e-05
15	2.40481567382813	2.40484619140625	2.40483093261719	-2.79037388226405e-05
16	2.40481567382813	2.40483093261719	2.40482330322266	1.17040462478698e-05
17	2.40482330322266	2.40483093261719	2.40482711791992	-8.09986199381796e-06
18	2.40482330322266	2.40482711791992	2.40482521057129	1.80208819959307e-06
19	2.40482521057129	2.40482711791992	2.40482616424561	-3.14888787994886e-06
20	2.40482521057129	2.40482616424561	2.40482616424561	-3.14888787994886e-06

Bisection yönteminin mutlak hatası bu şekilde hesaplanır

$$|p_n - p| \leq \frac{b_n - a_n}{2}$$

O zaman

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

$$|p_n - p| \leq 4.768371582031250e-07$$

3.2)Fixed point yöntemi ile (p0 = 2.5 alındı tahmini aralığın yarısı(g şıkkına bakınız))

n	p_n	$f(p)_n$
0	2.50000000000000	-0.483837764681979
1	-0.483837764681979	9.42325992933161
2	9.42325992933161	-1.80942948791555
3	-1.80942948791555	3.34502309172189
4	3.34502309172189	-3.53813386670414
5	-3.53813386670414	-3.85061519836204
6	-3.85061519836204	-4.02687510748892
7	-4.02687510748892	-3.95237940579397
8	-3.95237940579397	-3.99860477518174
9	-3.99860477518174	-3.97241584597260
10	-3.97241584597260	-3.98826205546148
11	-3.98826205546148	-3.97898758587437
12	-3.97898758587437	-3.98453268999997
13	-3.98453268999997	-3.98125694733214
14	-3.98125694733214	-3.98320632965977
15	-3.98320632965977	-3.98205121429512
16	-3.98205121429512	-3.98273744191241
17	-3.98273744191241	-3.98233038646896
18	-3.98233038646896	-3.98257206081466
19	-3.98257206081466	-3.98242865207945
20	2.50000000000000	-0.483837764681979

İstediğimiz aralığın dışına değerler çıktı ve kök bulunamadı

3.3) Newton raphson metodu ile (p0 = 2.5 alındı tahmini aralığın yarısı)

n	p_n	$f(p)_n$
0	2.50000000000000	-0.483837764681979
1	2.40266676625544	0.0112123363919162
2	2.40482459179194	5.01446659545133e-06
3	2.40482555769558	1.00708169328851e-12
4	2.40482555769577	0

$$R(p_n) = \frac{f''(\varepsilon)}{2} * (p_n - p_{n-1})^2$$

$$R(p_n) = -\frac{22376792575 * x^3}{75703688197437786823974636480028999 68} + \frac{2404108741 * x^2}{42057604554132103791097020266682777 6} + \frac{18287000471 * x}{23365335863406724328387233481490432 0} - \frac{245843751181}{17524001897555043246290425111117824 00}$$

Ve

$$\max\big(R(p)\big) = \quad 4.491549758320938e - 26$$

3.4)Sekant metodu ile (p0 =2 , p1 = 3 alındı, tahmini aralığın sınırları)

n	p_n	$f(p)_n$
0	2	2.23890779141236
1	3	-2.60051954901934
2	2.46263899298727	-0.296420873174667
3	2.39350783095040	0.0588931328735816
4	2.40496628922179	-0.000730582815975592
5	2.40482588615207	-1.70517254195778e-06
6	2.40482555768616	4.98985125661385e-11
7	2.404825557695772	1.57274600722102e-15
8	2.404825557695773	0

Secant yönteminin hatası ε hata olmak üzeri bu şekide hesaplanabilir

$$|r_{n+1} - r_n| \leq \varepsilon$$

O zaman $\varepsilon = \pm 1.57274600722102\text{e} - 15$

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

2.kök için kod:

```
xvalues = [0 1.2 2.4 3.6 4.8 6];
yvalues=[10.0000 6.7113 0.0251 -3.9177 -2.4043 1.5065];
lpoli = lagrangeip(xvalues, yvalues , 5);
npoli =newtonip(xvalues,yvalues,5);
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
lpoli_f = matlabFunction(lpoli);
[l_a_it_b,l_b_it_b,l_p_it_b,l_fp_it_b] = bisection(lpoli_f,[5,6],20);
lpoli_f_bis_t = table(l_a_it_b.',l_b_it_b.',l_p_it_b.',l_fp_it_b.');
l_poli_bis_err  =( l_b_it_b(end) - l_a_it_b(end)) / 2;

[l_p_it_f , l_fp_it_f] = fixed_point(lpoli_f,5.5,20);
lpoli_f_fixed_t = table(l_p_it_f.',l_fp_it_f.');

[l_p_it_n , l_fp_it_n] = newton_raphson(lpoli,5.5,6);
lpoli_f_newtonr_t = table(l_p_it_n.',l_fp_it_n.');
[l_raph_error ,l_raph_error_func] = calculate_newton_raphson_error...
(lpoli , l_p_it_n(end-1) , l_p_it_n(end), [5,6]);

[l_p_it_s , l_fp_it_s] = secant_method(lpoli_f,5,6,20);
lpoli_f_secant_t = table(l_p_it_s.',l_fp_it_s.');
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
npoli_f = matlabFunction(npoli);
[n_a_it_b,n_b_it_b,n_p_it_b,n_fp_it_b] = bisection(npoli_f,[5,6],20);
npoli_f_bis_t = table(n_a_it_b.',n_b_it_b.',n_p_it_b.',n_fp_it_b.');
npoli_bis_err  =( n_b_it_b(end) - n_a_it_b(end)) / 2;

[n_p_it_b , n_fp_it_b] = fixed_point(npoli_f,5.5,20);
npoli_f_fixed_t = table(n_p_it_b.',n_fp_it_b.');

[n_p_it_n , n_fp_it_n] = newton_raphson(npoli,5.5,4);
npoli_f_newtonr_t = table(n_p_it_n.',n_fp_it_n.');
[n_raph_error ,n_raph_error_func] = calculate_newton_raphson_error...
    (lpoli , n_p_it_n(end-1) , n_p_it_n(end), [5,6]);

[n_p_it_s , n_fp_it_s] = secant_method(npoli_f,5,6,20);
npoli_f_secant_t = table(n_p_it_s.',n_fp_it_s.');
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
syms x real;
fx_poli = 10 * besselj(0,x);
fx_poli_f = matlabFunction(fx_poli);

[f_a_it_b,f_b_it_b,f_p_it_b,f_fp_it_b] = bisection(fx_poli_f,[5,6],20);
fxpoli_f_bis_t = table(f_a_it_b.',f_b_it_b.',f_p_it_b.',f_fp_it_b.');
fx_poli_bis_err  =( f_b_it_b(end) - f_a_it_b(end)) / 2;

[p_it , fp_it] = fixed_point(fx_poli_f,5.5,20);
fxoli_f_fixed_t = table(p_it.',fp_it.');

[p_it , fp_it] = newton_raphson(fx_poli,5.5,3);
fxpoli_f_newtonr_t = table(p_it.',fp_it.');
[fx_raph_error ,fx_raph_error_func] = calculate_newton_raphson_error(...
   lpoli , p_it(end-1) , p_it(end), [5,6]);

[p_it , fp_it] = secant_method(fx_poli_f,5,6,9);
fxpoli_f_secant_t = table(p_it.',fp_it.');
```

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

2.kök için

1)Lagrange interpolasyon polinomu için

1.1)Bisection yöntemi ile kök bulma :

n	a_n	b_n	p_n	$f(p)_n$
0	5	6	5.50000000000000	0.0904909141341719
1	5	5.50000000000000	5.25000000000000	-0.819895986557000
2	5.25000000000000	5.50000000000000	5.37500000000000	-0.359068486954541
3	5.37500000000000	5.50000000000000	5.43750000000000	-0.132116637321182
4	5.43750000000000	5.50000000000000	5.46875000000000	-0.0201706275674027
5	5.46875000000000	5.50000000000000	5.48437500000000	0.0353333375714016
6	5.46875000000000	5.48437500000000	5.47656250000000	0.00762306351271036
7	5.46875000000000	5.47656250000000	5.47265625000000	-0.00626355272949297
8	5.47265625000000	5.47656250000000	5.47460937500000	0.000682337403429045
9	5.47265625000000	5.47460937500000	5.47363281250000	-0.00278996524857789
10	5.47363281250000	5.47460937500000	5.47412109375000	-0.00105365293307358
11	5.47412109375000	5.47460937500000	5.47436523437500	-0.000185617469192323
12	5.47436523437500	5.47460937500000	5.47448730468750	0.000248370047067681
13	5.47436523437500	5.47448730468750	5.47442626953125	3.13788081598432e-05
14	5.47436523437500	5.47442626953125	5.47439575195313	-7.71187008048457e-05
15	5.47439575195313	5.47442626953125	5.47441101074219	-2.28697888573493e-05
16	5.47441101074219	5.47442626953125	5.47441864013672	4.25454900820910e-06
17	5.47441101074219	5.47441864013672	5.47441482543945	-9.30761010309311e-06
18	5.47441482543945	5.47441864013672	5.47441673278809	-2.52652807830600e-06
19	5.47441673278809	5.47441864013672	5.47441768646240	8.64011077794658e-07
20	5.47441673278809	5.47441768646240	5.47441768646240	8.64011077794658e-07

Bisection yönteminin mutlak hatası bu şekilde hesaplanır

$$|p_n - p| \leq \frac{b_n - a_n}{2}$$

O zaman

$$|p_n - p| \leq 4.768371582031250e-07$$

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

1.2)Fixed point yöntemi ile (p0 = 2.5 alındı tahmini aralığın yarısı(g şıkkına bakınız))

n	p_n	$f(p)_n$
0	5.50000000000000	0.0904909141341719
1	0.0904909141341719	10.0337590913431
2	10.0337590913431	-199.983208650812
3	-199.983208650812	2545183796.57242
4	2545183796.57242	-8.38249358265348e+44
5	-8.38249358265348e+44	3.24820648941753e+222
6	3.24820648941753e+222	NaN

Bir noktaya yaklaşmak söz konusu olmadığından kök bulunamadı.

1.3)Newton raphson metodu ile (p0 = 2.5 alındı tahmini aralığın yarısı)

n	p_n	$f(p)_n$
0	5.50000000000000	0.0904909141341719
1	5.47428106939667	-0.000484854701408466
2	5.47441743990191	-1.25700125863659e-08
3	5.47441744343755	-2.84217094304040e-14
4	5.47441744343755	2.13162820728030e-14
5	5.47441744343755	-7.10542735760100e-15
6	5.47441744343755	3.55271367880050e-15

Newton metodunun hata payı ,
epsilon tahmini kök aralığında([2.4,2.5])(fonksiyon aralığı almamıza gerek yoktur çünkü p0 = 2.5 verdik ve p_n 2.4 ve 2.5 aralığında olduğunu görüyoruz) bir sayı olmak üzere şu şekilde bulunur

$$R(p_n) = \frac{f''(\varepsilon)}{2} * (p_n - p_{n-1})^2$$

Bu hatayı bulmak için calculate_newton_raphson_error adında bir fonksiyon yazıldı içinde R(p) hata polinomunu bulundu sonra aralık sınırları içinde delta = 0.0001 olmak üzere epsilon örneklenip R(x)'in maximum olduğu nokta bulundu.

$$R(p) = -\frac{117175 * x^3}{473148051233986167649841478000181248} + \frac{12589 * x^2}{26286002846332564869435637666676736} + \frac{95759 * x}{146033349146292027052420209259315 20} - \frac{1287349}{109525011859719020289315156944486400}$$

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

$$\max\big(R(p)\big) = \quad 2.049644540373287e-30$$

1.4)Sekant metodu ile (p0 =2 , p1 = 3 alındı, tahmini aralığın sınırları)

n	p_n	$f(p)_n$
0	5	-1.73023905017574
1	6	1.50650000000001
2	5.53456241709748	0.211140499852434
3	5.45869720447276	-0.0560520146994605
4	5.47461231354814	0.000692783871489411
5	5.47441800973249	2.01331292615237e-06
6	5.47441744341654	-7.46780415283865e-11
7	5.47441744343755	3.55271367880050e-15
8	5.47441744343755	1.06581410364015e-14
9	5.47441744343755	3.55271367880050e-15
10	5.47441744343755	3.55271367880050e-15

Secant yönteminin hatası ε hata olmak üzeri bu şekide hesaplanabilir

$$|r_{n+1} - r_n| \leq \varepsilon$$

O zaman $\varepsilon = \pm \mathrm{e} - 16$

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

2)Newton interpolasyon polinomu için

2.1) Bisection yöntemi ile kök bulma :

n	a_n	b_n	p_n	$f(p)_n$
0	5	6	5.50000000000000	0.0904909141341683
1	5	5.50000000000000	5.25000000000000	-0.819895986556990
2	5.25000000000000	5.50000000000000	5.37500000000000	-0.359068486954527
3	5.37500000000000	5.50000000000000	5.43750000000000	-0.132116637321186
4	5.43750000000000	5.50000000000000	5.46875000000000	-0.0201706275673850
5	5.46875000000000	5.50000000000000	5.48437500000000	0.0353333375714051
6	5.46875000000000	5.48437500000000	5.47656250000000	0.00762306351271391
7	5.46875000000000	5.47656250000000	5.47265625000000	-0.00626355272947521
8	5.47265625000000	5.47656250000000	5.47460937500000	0.000682337403432598
9	5.47265625000000	5.47460937500000	5.47363281250000	-0.00278996524856723
10	5.47363281250000	5.47460937500000	5.47412109375000	-0.00105365293305582
11	5.47412109375000	5.47460937500000	5.47436523437500	-0.000185617469174559
12	5.47436523437500	5.47460937500000	5.47448730468750	0.000248370047078339
13	5.47436523437500	5.47448730468750	5.47442626953125	3.13788081705013e-05
14	5.47436523437500	5.47442626953125	5.47439575195313	-7.71187007941876e-05
15	5.47439575195313	5.47442626953125	5.47441101074219	-2.28697888751128e-05
16	5.47441101074219	5.47442626953125	5.47441864013672	4.25454902597267e-06
17	5.47441101074219	5.47441864013672	5.47441482543945	-9.30761009243497e-06
18	5.47441482543945	5.47441864013672	5.47441673278809	-2.52652808185871e-06
19	5.47441673278809	5.47441864013672	5.47441768646240	8.64011095558226e-07
20	5.47441673278809	5.47441768646240	5.47441768646240	8.64011095558226e-07

Bisection yönteminin mutlak hatası bu şekilde hesaplanır

$$|p_n - p| \leq \frac{b_n - a_n}{2}$$

O zaman

$$|p_n - p| \leq 4.768371582031250e-07$$

2.2)Fixed point yöntemi ile (p0 = 5 .5 alındı tahmini aralığın yarısı(g şıkkına bakınız))

n	p_n	$f(p)_n$
0	5.50000000000000	0.0904909141341683
1	0.0904909141341683	10.0337590913431
2	10.0337590913431	-199.983208650812
3	-199.983208650812	2545183796.57240
4	2545183796.57240	-8.38249358265327e+44
5	-8.38249358265327e+44	3.24820648941713e+222
6	3.24820648941713e+222	NaN

Bir noktaya yaklaşmak söz konusu olmadığından kök bulunamadı.

2.3) Newton raphson metodu ile (p0 = 5.5 alındı tahmini aralığın yarısı)

n	p_n	$f(p)_n$
0	5.50000000000000	0.0904909141341683
1	5.47428106939667	-0.000484854701383597
2	5.47441743990191	-1.25700267972206e-08
3	5.47441744343754	-1.42108547152020e-14
4	5.47441744343755	1.77635683940025e-14

$$R(p_n) = \frac{f''(\varepsilon)}{2} * (p_n - p_{n-1})^2$$

$$R(p) = -\frac{2929375 * x^3}{189259220493594467059936591200072499 2} + \frac{314725 * x^2}{105144011385330259477742550666706944} + \frac{478795 * x}{116826679317033621641936167407452 16} - \frac{1287349}{175240018975550432462904251111178 24}$$

Ve

$$\max\big(R(p)\big) = \ 1.281027837733303e - 29$$

2.4)Sekant metodu ile (p0 =5 , p1 = 6 alındı, tahmini aralığın sınırları)

n	p_n	$f(p)_n$
0	5	-1.73023905017575
1	6	1.50650000000002
2	5.53456241709748	0.211140499852448
3	5.45869720447275	-0.0560520146994676
4	5.47461231354813	0.000692783871485858
5	5.47441800973249	2.01331289417794e-06
6	5.47441744341655	-7.46638306736713e-11
7	5.47441744343755	0

Secant yönteminin hatası ε hata olmak üzeri bu şekide hesaplanabilir

$$|r_{n+1} - \ r_n| \leq \ \varepsilon$$

O zaman $\varepsilon = 3.54$ e $- 12$

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

3) f(x) polinomu için

3.1) Bisection yöntemi ile kök bulma :

n	a_n	b_n	p_n	$f(p)_n$
0	5	6	5.50000000000000	-0.0684386941781923
1	5.50000000000000	6	5.75000000000000	0.759753320169011
2	5.50000000000000	5.75000000000000	5.62500000000000	0.353013198107559
3	5.50000000000000	5.62500000000000	5.56250000000000	0.143751807320986
4	5.50000000000000	5.56250000000000	5.53125000000000	0.0379748031886749
5	5.50000000000000	5.53125000000000	5.51562500000000	-0.0151584320854015
6	5.51562500000000	5.53125000000000	5.52343750000000	0.0114273225631227
7	5.51562500000000	5.52343750000000	5.51953125000000	-0.00186086517975801
8	5.51953125000000	5.52343750000000	5.52148437500000	0.00478441293112698
9	5.51953125000000	5.52148437500000	5.52050781250000	0.00146206845566525
10	5.51953125000000	5.52050781250000	5.52001953125000	-0.000199324902092810
11	5.52001953125000	5.52050781250000	5.52026367187500	0.000631390164909089
12	5.52001953125000	5.52026367187500	5.52014160156250	0.000216037225546853
13	5.52001953125000	5.52014160156250	5.52008056640625	8.35730990114335e-06
14	5.52001953125000	5.52008056640625	5.52005004882813	-9.54835090959215e-05
15	5.52005004882813	5.52008056640625	5.52006530761719	-4.35630278419299e-05
16	5.52006530761719	5.52008056640625	5.52007293701172	-1.76028410302648e-05
17	5.52007293701172	5.52008056640625	5.52007675170898	-4.62276108188307e-06
18	5.52007675170898	5.52008056640625	5.52007865905762	1.86727553214875e-06
19	5.52007675170898	5.52007865905762	5.52007770538330	-1.37774249599332e-06
20	5.52007770538330	5.52007865905762	5.52007770538330	-1.37774249599332e-06

Bisection yönteminin mutlak hatası bu şekilde hesaplanır

$$|p_n - p| \leq \frac{b_n - a_n}{2}$$

O zaman

$$|p_n - p| \leq 4.768371582031250e-07$$

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

3.2)Fixed point yöntemi ile (p0 = 2.5 alındı tahmini aralığın yarısı(g şıkkına bakınız))

n	p_n	$f(p)_n$
0	5.50000000000000	-0.0684386941781923
1	-0.0684386941781923	9.98829379029206
2	9.98829379029206	-2.45409719565079
3	-2.45409719565079	-0.253104865143048
4	-0.253104865143048	9.84048492020945
5	9.84048492020945	-2.35835818098580
6	-2.35835818098580	0.243507664944676
7	0.243507664944676	9.85230851585885
8	9.85230851585885	-2.36800092697162
9	-2.36800092697162	0.192609432787469
10	0.192609432787469	9.90746883996365
11	9.90746883996365	-2.40844849773075
12	-2.40844849773075	-0.0187942079143780
13	-0.0187942079143780	9.99911696386667
14	9.99911696386667	-2.45897278868287
15	-2.45897278868287	-0.277850211253402
16	-0.277850211253402	9.80792739850549
17	9.80792739850549	-2.33004406228526
18	-2.33004406228526	0.394023124176986
19	0.394023124176986	9.61561447131022
20	5.50000000000000	-0.0684386941781923

İstediğimiz aralığın dışına değerler çıktı ve kök bulunamadı

3.3) Newton raphson metodu ile (p0 = 2.5 alındı tahmini aralığın yarısı)

n	p_n	$f(p)_n$
0	5.50000000000000	-0.0684386941781923
1	5.52004423965613	-0.000115250187844122
2	5.52007811018241	-3.53534479359273e-10
3	5.52007811028631	7.22802933433496e-16

$$R(p_n) = \frac{f^{"}(\varepsilon)}{2} * (p_n - p_{n-1})^2$$

$$R(p_n) = -\frac{1603487657250175 * x^3}{1892592204935944670599365912000724992} + \frac{172274854850629 * x^2}{10514401138533025947774255066670694 4} + \frac{1310419241054999 * x}{584133396585168108209680837037260 80} - \frac{17616797372078989}{438100047438876081157260627777945600}$$

Ve

$$\max\big(R(p)\big) = \ 7.012118033366270e - 21$$

3.4)Sekant metodu ile (p0 =2 , p1 = 3 alındı, tahmini aralığın sınırları)

n	p_n	$f(p)_n$
0	5	-1.77596771314338
1	6	1.50645257250997
2	5.54105433143516	0.0712342053162384
3	5.51827547719887	-0.00613472439632757
4	5.52008165434677	1.20591865834734e-05
5	5.52007811086310	1.96261177793863e-09
6	5.52007811028631	7.22802933433496e-16
7	5.52007811028631	7.22802933433496e-16

Secant yönteminin hatası ε hata olmak üzeri bu şekide hesaplanabilir

$$|r_{n+1} - \ r_n| \leq \ \varepsilon$$

O zaman $\varepsilon = \ \pm \mathrm{e} - 16$

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

i)**1.kök için ([2,3] aralığında):**

Fonksiyon	Bisection metodu	Fixed point metodu	Newton-raphson metodu	Secant metodu
Lagrange interpolasyon polinomu	p20 = 2.40491771697998 tolerans = 4.768371582031250e-07	Doğru kök bulunamadı (h' a bakınız)	p6 = 2.404918190460423 tolerans =2.3520e-31	p8= 2.404838759700320 tolerans = ± 7.1e-15
Newton interpolasyon polinomu	p20 =2.40491771697998 tolerans = 4.768371582031250e-07	Doğru kök bulunamadı	p4 = 2.40491819046042 tolerans = 3.934413099098205e-26	p9 =2.404918190460422287766 tolerans = ±5.329070518200775e-15
f(x) polinomu	p20 = 2.40482521057129 tolerans =4.768371582031250e-07	Doğru kök bulunamadı	p4=2.40482555769577 tolerans = 4.491549758320938e-26	p8=2.404825557695773 tolerans = 1.57274600722102e-15

2.kök için ([5,6] aralığında):

Fonksiyon	Bisection metodu	Fixed point metodu	Newton-raphson metodu	Secant metodu
Lagrange interpolasyon polinomu	p20 = 5.47441673278809 tolerans = 4.768371582031250e-07	Doğru kök bulunamadı (h' a bakınız)	p6 = 5.47441744343755 tolerans =2.049644540373287e-30	p10= 5.47441744343755 tolerans = ± e-16
Newton interpolasyon polinomu	p20 =5.47441673278809 tolerans = 4.768371582031250e-07	Doğru kök bulunamadı	p4 = 5.47441744343755 tolerans = 1.281027837733303e-29	p7 =5.4744174434375 5 tolerans = ± 3.54 e-12
f(x) polinomu	p20 = 5.52007770538330 tolerans =4.768371582031250e-07	Doğru kök bulunamadı	p3=5.52007811028631 tolerans = 7.012118033366270e-21	p7=5.52007811028631 tolerans = ± e-16

Yukardaki tablodan Newton metodunun kökü en hızlı şekilde bulduğunu görebiliriz , mesela newtonun dördüncü iterasyonunda bulunan kök secant metodu ile 8.inci iterasyonda bulunmaktadır, bisection metodu ise daha fazla iterasyonda bulmaktadır. Ama unutulmaması gereken bir şey var Newton metodun da p0'a 2.5 değeri verildi kökten daha uzak bir değer verilmiş olasaydı daha fazla iterasyona ihtiyaç duyardı, ve bisectionda eğer fonksiyon aralığı değil de daha fazla köke yakın aralıkta bir aralık verilseydi daha hızlı bulabilirdi, yine de Newton-raphsonun yaklaşma hızı sekant yönteminden büyüktür, sekant yönteminin yaklaşma hızı ise bisectiondan büyüktür.
Sabit nokta metoduna gelince fonksiyonda bir sabit noktaya yaklaşma olmadı onun için kök bulunamadı, böylece sabit nokta iterasyonun sadece spesifik problemlerde uygulanabileceğini görürüz ve bizim fonksiyonumuzda kullanılmaya uygun değil.
j)Lagrange interpolasyonu ve Newton interpolasyonu ile fonksiyon oturturmayı öğrendim ve bu iki metodun aynı sonucu verdiğini gözlemledim.
Ayrıca kök bulma metodların avantajları ve dezavantajlarını öğrendim:
Newton metodunun avantajı köke hızlı yaklaşmasıdır buna karşın isterleri bir başlangıç noktası ve fonksiyonun türevi , her zaman fonksiyonun türevini bulmak mümkün olmayabilir bunun için nümerik türev kullanılabilir , bu ihtiyacı kaldırmak için sekant yöntemi geliştirilmiş ve fonksiyonun türevini kullanmadan bir birine yakın iki nokta seçerek (p0 , p1) kökün bulunması sağlanmaktadır buradaki ihtiyacımız ise ikinci bir noktadır ve genellikle bu ihtiyacı sağlamak mümkündür. Bisection'da ise tahmini bir aralık vermekteyiz Newton ve sekant metodundan daha yavaş ama bir türeve ihtiyaç duyulmamaktadır, sekant yöntemiyle ile çözünemeyen problemler daha fazla işlem yüküne katlanarak bisection ile çözülebilir. Ayrıca MATLAB kullanmayı , fonksiyon , vektör ve tablo oluşturmayı ve onlarla işlem yapmayı öğrendim.